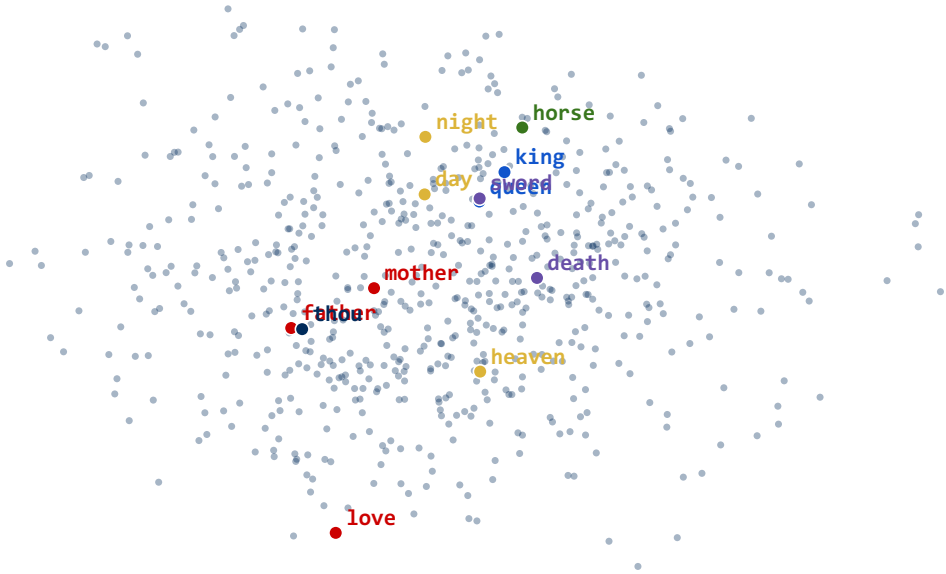
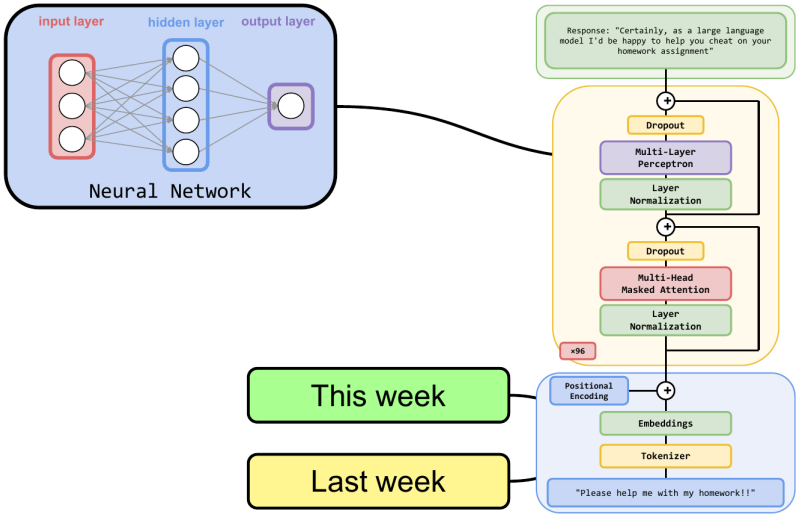


UNIT 01 · CHAPTER 6

Text Embeddings

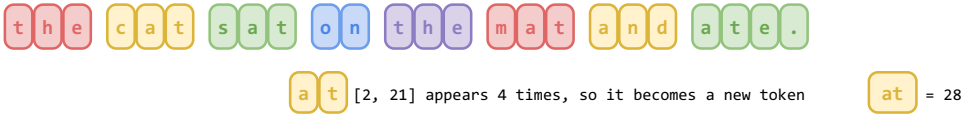
CSE 10124 · Building ChatGPT
William Theisen · University of Notre Dame

CSE 10124: Building ChatGPT



Training a BPE Tokenizer

Broadly speaking, we can train our **tokenizer** by **repeatedly looking for common pairings of tokens** and **merging them to create a new token**.



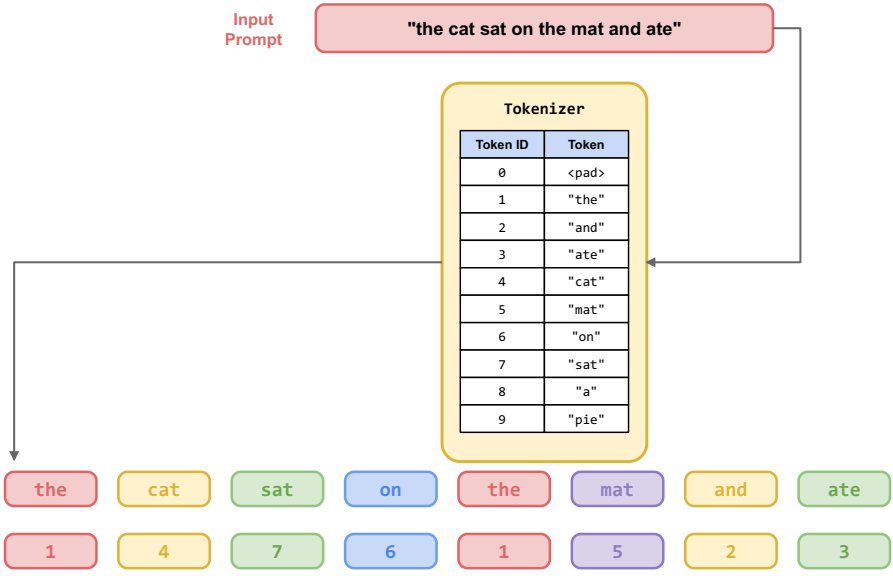

```
[21, 9, 6, 0, 4, 2, 21, 0, 20, 2, 21, 0,
16, 15, 0, 21, 9, 6, 0, 14, 2, 21, 0, 2,
15, 5, 0, 2, 21, 6, 1]
```

```
[2, 21]: 4 [21, 9]: 2 [9, 6]: 2 [4, 28]: 1
[20, 28]: 1 [16, 15]: 1 [14, 28]: 1 [2,
15]: 1 [15, 5]: 1 [28, 6]: 1
```

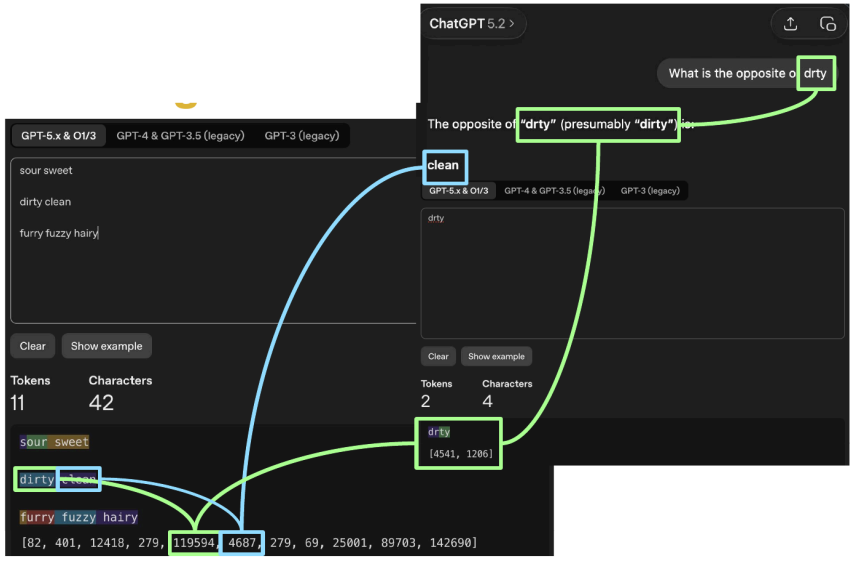
```
[21, 9, 6, 0, 4, 28, 0, 20, 28, 0, 16, 15,
0, 21, 9, 6, 0, 14, 28, 0, 2, 15, 5, 0, 28,
6, 1]
```

An untrained tokenizer is effectively just a character-level tokenizer.

Tokenizing a Prompt



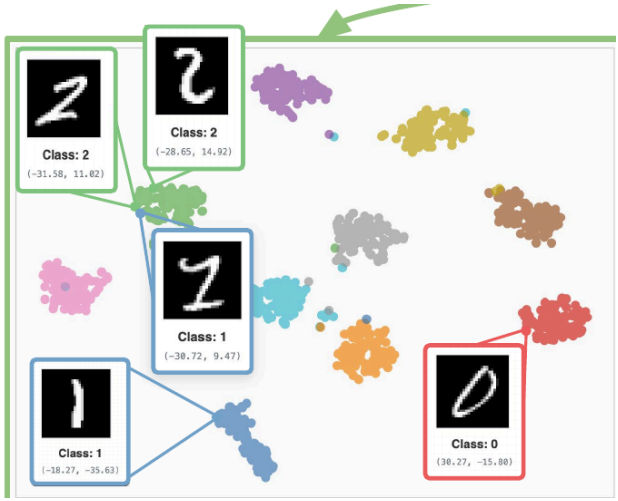
Tokenization vs. Meaning



To capture the **meaning** of **tokens**, we use something called **embeddings**

Image Embeddings

What is actually represented in the **<64, 256>** dimensional vector we get after the first layer?



The digit classifier, one equation per layer:

<64, 784> images, <784, 256> weights

$$Z_1 = X @ W_1^T + b_1$$

<64, 256>: the first layer's output

$$A_1 = \mathrm{ReLU}(Z_1)$$

<64, 10>

$$Z_2 = A_1 @ W_2^T + b_2$$

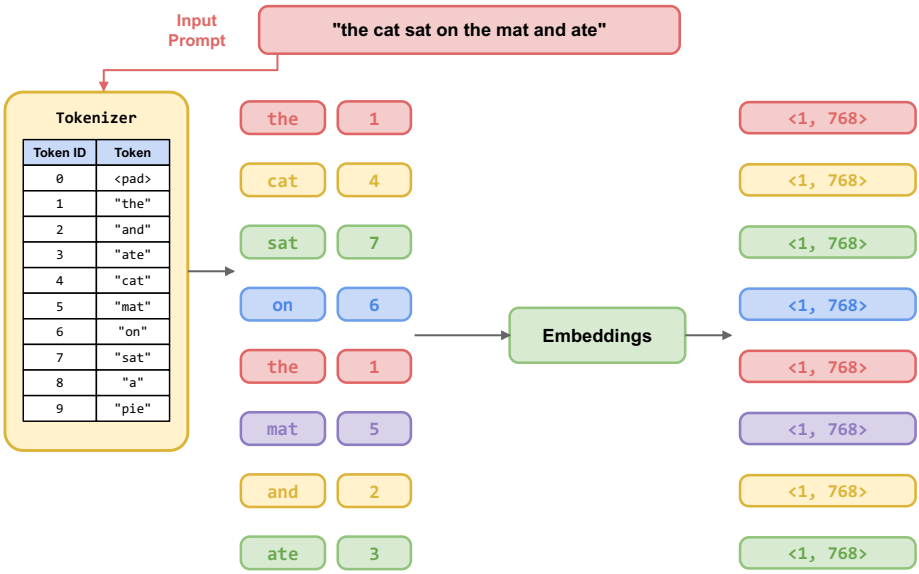
<64, 10>

$$\hat{y} = \mathrm{softmax}(Z_2)$$

against the ground truth vectors y, <64, 10>

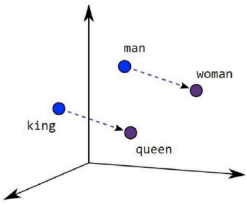
$$\mathcal{L}_{ce} = -\frac{1}{N}\sum_{i=1}^{N} y_i \cdot \log(\hat{y}_i)$$

Embeddings

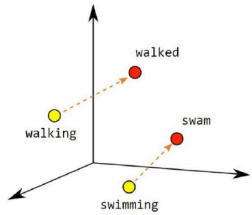

Input Prompt
"the cat sat on the mat and ate"
Tokenizer
Token ID | Token
0 | <pad>
1 | "the"
2 | "and"
3 | "ate"
4 | "cat"
5 | "mat"
6 | "on"
7 | "sat"
8 | "a"
9 | "pie"
the 1
cat 4
sat 7
on 6
the 1
mat 5
and 2
ate 3
Embeddings
<1, 768>
<1, 768>
<1, 768>
<1, 768>
<1, 768>
<1, 768>
<1, 768>
<1, 768>

Text Embeddings

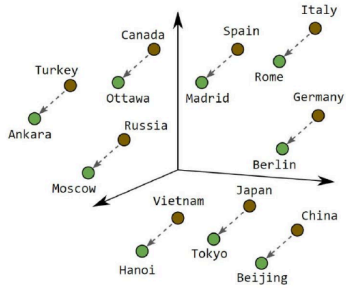
Text Embeddings



Male-Female



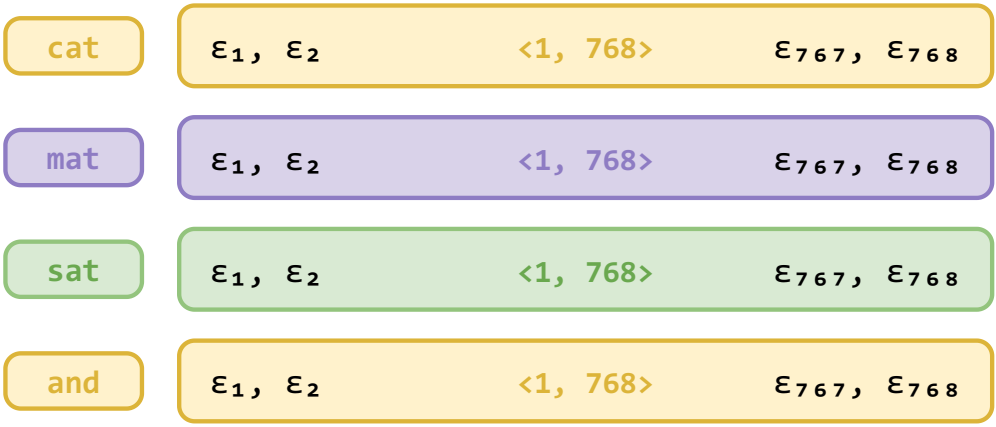
Verb Tense



Country-Capital

Converting **tokens** to **embeddings** allows us to **interpret them as points in high dimensional space**, which then allows us to compare them against each other!

Text Embeddings



One way to think about it, is that **each axis encodes some artifact of language**, plurality, color, gender…

How similar they are on a given axis measures how similar they are on that "thing".

> **NOTE:** Technically the similarity is often on a higher-dimensional manifold than a singular axis, but it's a good way to think about it

Axes in the Brain

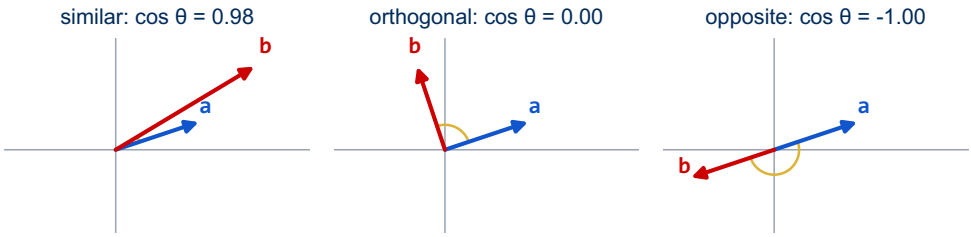
Mitchell et al. (*Science*, `2008`) scanned `9` people with fMRI while they looked at `60` nouns (celery is one).

- Each noun got `25` axes: how often it appears near `25` verbs (*eat*, *taste*, *push*, *ride*, …) in a trillion-word text corpus. Celery's *eat* value: `0.84`.
- A linear model from those `25` numbers to brain activity, fit on `58` nouns, told the `2` left-out nouns apart `77%` of the time (chance is `50%`).

Huth et al. (*Nature*, `2016`): `7` people, over `2` hours of *The Moth Radio Hour*, every word described by its co-occurrence with `985` common words. The result is a **map of meaning** across both sides of the cortex, broadly similar from person to person.

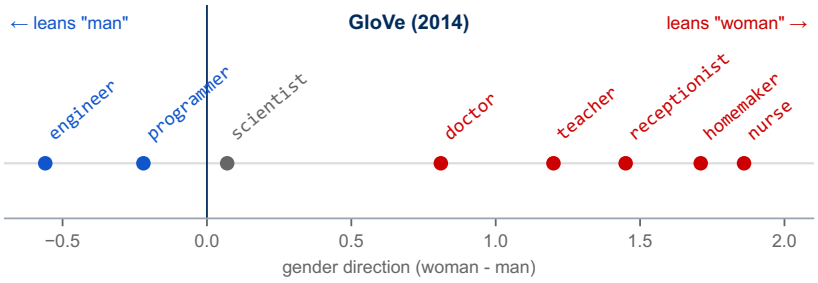
Embeddings built from co-occurrence can predict brain activity. They don't show the brain learns meaning by counting.

Measuring Text Embedding Similarity



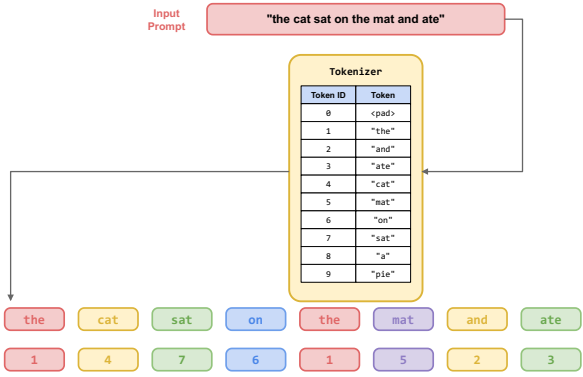
Instead of using **euclidean distance**, we often measure the similarity of text embeddings using **cosine similarity**, which measures the angle between two vectors. This is because we often **care about direction**, **rather than magnitude**.

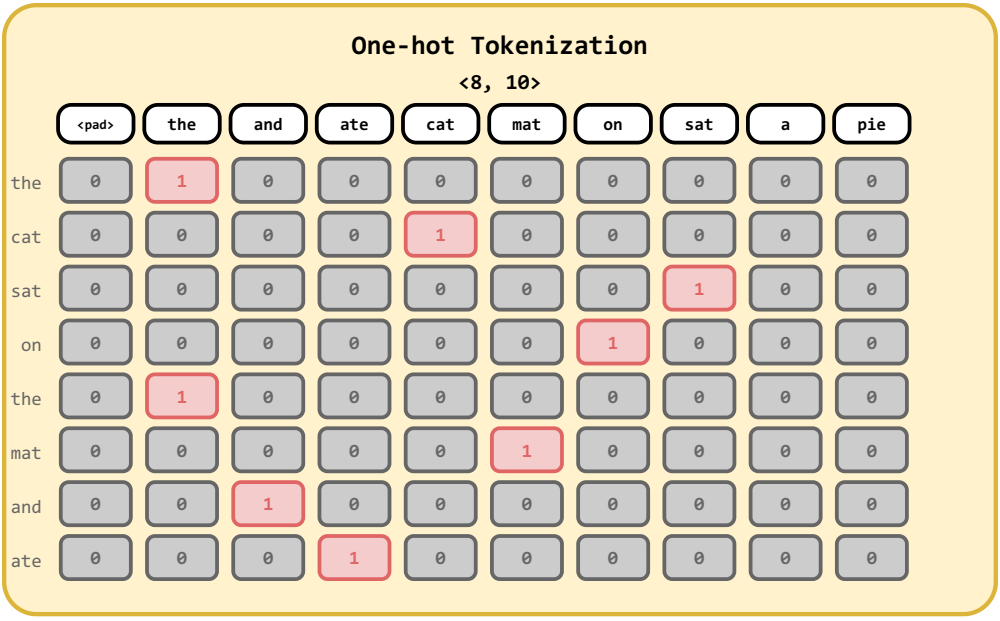
Text Embeddings: Notebook



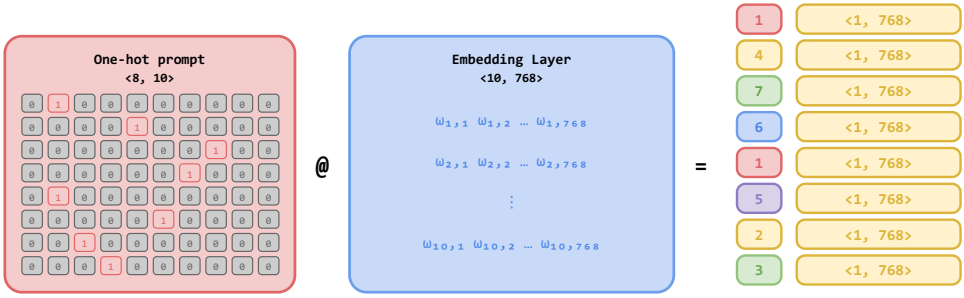
This allows us to measure how words are related to each other. **Remember that related is only based on co-occurrences!!**

One-Hot Tokenization

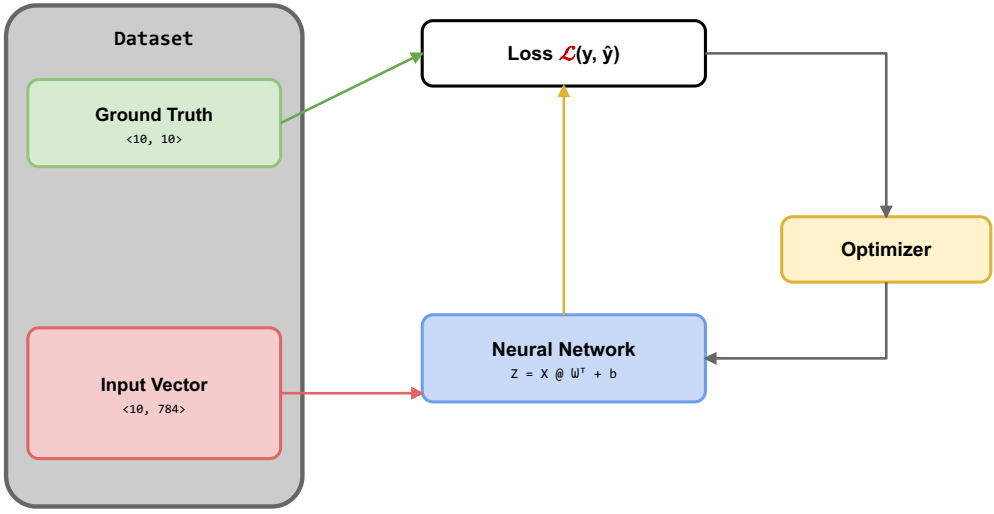




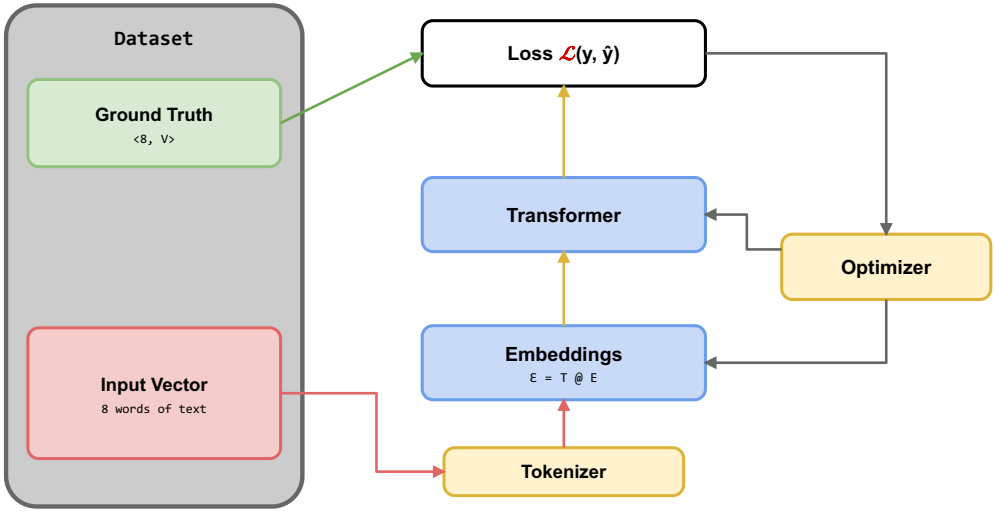
Producing Text Embeddings for an Input Prompt



Training an Image Classification Model



Training a Text Prediction Model



Next Class: Anthropic Guest Lecture

Noah Yoshida (Class of 2020), Member of Technical Staff, **Anthropic**

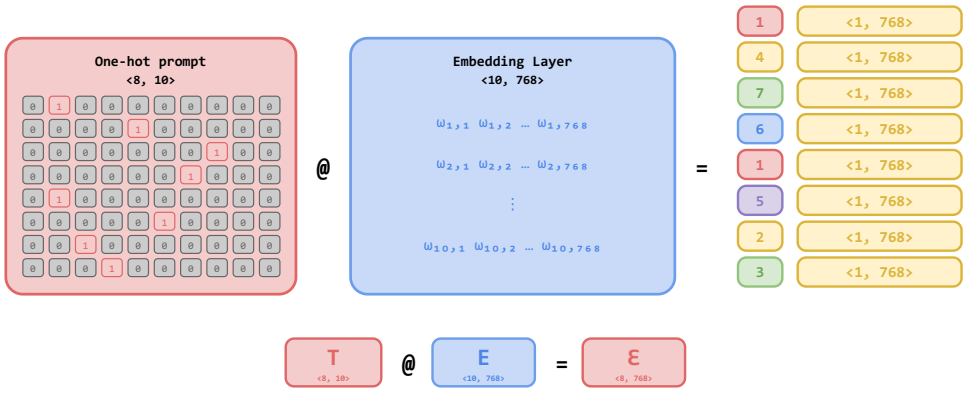
Wednesday September 16th. Ask me anything!

- Debart 214, **11:30 – 12:20**
- Debart 141, **3:30 – 4:45**

Programming Day 04: Text Embeddings

Today we write irishQwen's **EmbeddingLayer** (`irishQwen/embedding.py`):

- `__init__`: a weight matrix `W` of shape `(embed_dim, vocab_size)`, one column per token
- `forward`: look up the columns for a batch of token ids, `(B, T)` → `(B, T, C)`
- `backward`: add the gradient into the columns of the tokens that were used
- `update`: one step of gradient descent



EmbeddingLayer: forward

```python
import torch
import math
class EmbeddingLayer():
    def __init__(self, vocab_size, embed_dim):
        self.W = torch.randn(embed_dim, vocab_size) \
            * math.sqrt(2.0 / vocab_size)
        self.dW = torch.zeros_like(self.W)
    def forward(self, tokens):
        self.tokens = tokens
        E = self.W[:, tokens]            # (C, B, T)
        E = E.permute(1, 2, 0)           # (B, T, C)
        return E.contiguous()
torch.manual_seed(0)
layer = EmbeddingLayer(512, 256)
tokens = torch.tensor([[446, 309, 44, 32,
                        275, 327, 294, 309]])
out = layer.forward(tokens)
print(tokens.shape, "->", out.shape)
print(torch.equal(out[0, 1], out[0, 7]))   # " be" twice
```

OUTPUT

```
torch.Size([1, 8]) -> torch.Size([1, 8, 256])
tensor(True)
```

Indexing `W[:, tokens]` is the same as `T @ E` without building `T`.

EmbeddingLayer: backward and update

```python
import torch.nn.functional as F

def backward(self, dY):
    C, V = self.W.shape
    flat = self.tokens.reshape(-1)          # (B*T,)
    dY_flat = dY.reshape(-1, C)             # (B*T, C)
    T = F.one_hot(flat, V).float()          # (B*T, V)
    self.dW = dY_flat.T @ T                 # (C, V)

def update(self, lr):
    self.W -= lr * self.dW

EmbeddingLayer.backward = backward
EmbeddingLayer.update = update
layer.backward(torch.ones(1, 8, 256))
used = (layer.dW[0] != 0).nonzero().reshape(-1)
print(used)
print(layer.dW[0, 309], layer.dW[0, 446])
```

OUTPUT

```
torch.Size([1, 8]) -> torch.Size([1, 8, 256])
tensor(True)
tensor([ 32,  44, 275, 294, 309, 327, 446])
tensor(2.) tensor(1.)
```

Only the columns of tokens in the batch get a gradient, and " be" (309), used twice, gets twice as much.

Training a Text Prediction Model

